TerraLens

Classical-CV Land-Cover Classification and Change Analyzer for Multi-Date Satellite Imagery

Build Your Own Project (BYOP) - Project Report

Course	Computer Vision
Student	Anant Singh
Program	B.Tech Computer Science Engineering, VIT Bhopal University
Registration No.	24BAI10345
Repository	https://github.com/heyanantsingh/TerraLens

1. Introduction

TerraLens compares two RGB images of the same area taken on different dates and reports what changed into what. It uses only classical computer-vision techniques - registration, denoising, clustering, morphology, the Hough transform and colour-difference thresholding - so it needs no training labels and no GPU. The output is a land-cover map for each date (bare soil, vegetation, water, built-up), a building count, road lines, a change mask, and a from-to transition table in hectares.

The project is a demonstration and learning implementation. Its quantitative results are measured on procedurally generated scenes with exact ground truth (Section 16); it has **not** been validated on real satellite imagery.

2. Problem Statement

Change monitoring from satellite images is usually done with labelled datasets and deep networks, or by eye. Two practical problems recur. First, two acquisitions are never perfectly registered or equally illuminated, so plain image differencing mostly flags registration error and brightness drift. Second, labelled data and GPUs are often unavailable. TerraLens addresses both with a label-free pipeline that first aligns and radiometrically normalises the pair and only then compares land cover.

3. Objectives

- Register two dates and correct brightness/contrast differences.
- Classify each date into four land-cover classes without labels.
- Count buildings and detect road lines from the built-up class.
- Detect change robustly and report transitions in hectares.
- Measure every claim against ground truth and against a naive baseline.
- Provide tests, documentation and a reproducible CLI.

4. Functional Requirements

Module	Requirement
Input	Read two images (or generate the synthetic pair); exit with a clear message if an image cannot be read; resize date 2 if shapes differ.
Registration	Estimate translation with ECC and warp date 2 onto date 1.
Normalisation	Match per-channel mean/std of date 2 to date 1 using only pixels that already look similar.
Classification	Bilateral denoise, K-Means (k=8) in Lab space, name clusters with colour rules, majority-filter the map.

Module	Requirement
Object analysis	Buildings: morphological opening + connected components + rectangularity filter. Roads: thin remainder + probabilistic Hough.
Change	Class-transition mask gated by Lab Delta-E, cleaned with opening and a minimum-area filter.
Output	Five figures, analysis_report.json, transition table in hectares (user-supplied ground sample distance).

5. Non-Functional Requirements

Requirement	How it is addressed
CPU only	No deep learning; one 512x512 pair processes in about 0.2 s on the build machine (CPU, single run).
Determinism	All random seeds fixed; scene generation, K-Means and benchmark are repeatable.
Maintainability	One concern per module in src/; 17 automated tests.
Graceful failure	Unreadable input exits with an error message instead of a traceback.

6. Technology Stack

Technology	Role
Python 3.12	Implementation language
OpenCV	Bilateral filter, ECC, morphology, connected components, Hough, Otsu, colour conversion
NumPy	Array maths, transition matrices
scikit-learn	K-Means clustering
Matplotlib	Figures
Pytest	Automated tests
ReportLab	This PDF (built by docs/build_report.py from measured JSON)

7. System Architecture

Figure 1 shows the pipeline. Each box is one function or module in src/.

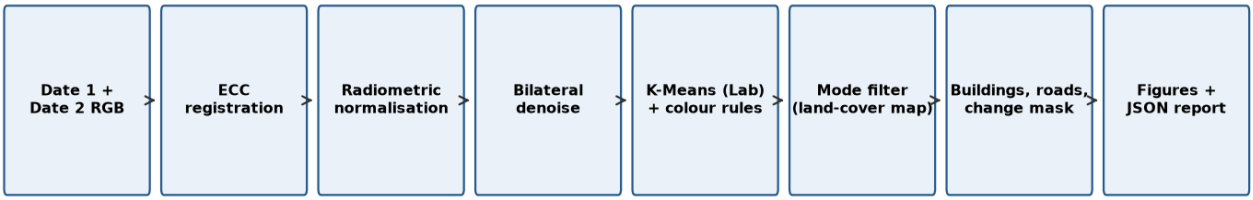


Figure 1. TerraLens pipeline

8. Design Decisions and Rationale

Register, then normalise, then classify. ECC maximises an enhanced correlation coefficient that is invariant to gain and bias, so it can register images whose brightness differs. Normalisation follows, using only below-median-difference pixels so that real change does not bias the correction.

K-Means in Lab with rule-based naming. Clustering needs no labels; naming clusters with three visible-band cues (greenness, blue dominance, low chroma) turns anonymous clusters into classes. Lab features are weighted (L x 0.6, a and b x 2.0) so hue matters more than brightness. Without this, a light-grey building cluster was merged with soil on one development seed (Section 18).

Two-condition change gate. A pixel counts as changed only if its class changed *and* its Lab colour difference exceeds half the Otsu threshold. This suppresses boundary flicker where the classifier disagrees between dates but the colour did not really change.

Rectangularity filter for buildings. A road junction survives morphological opening as a plus-shaped blob and was counted as a building. Requiring extent (area / bounding-box area) of at least 0.8 removes it, because rectangles score about 1.0 and the junction about 0.65.

Synthetic benchmark. Real labelled before/after pairs were not available offline. Generated scenes give exact labels so that accuracy can be measured rather than asserted.

9. Design Diagrams

Workflow, use-case, sequence and class/component diagrams are in the docs/ folder as Mermaid files that GitHub renders. Summary of the use cases:

Actor	Use case	System response
Analyst	Run demo (--demo)	Generate synthetic pair, run pipeline, write outputs
Analyst	Analyse own pair (--before --after --gsd)	Align, classify, report change in hectares
Analyst	Open analysis_report.json	Areas, transitions, building counts
Developer	Run pytest	17 automated tests

10. Implementation Details

Path	Purpose
main.py	CLI entry point
src/synthetic.py	Scene generator with exact ground truth
src/preprocess.py	Denoise, ECC alignment, radiometric normalisation
src/classify.py	Lab K-Means, rule naming, mode filter, baseline
src/features.py	Building blobs, road lines
src/change.py	Change detection, transitions, naive baseline
src/evaluate.py	Accuracy, IoU, precision, recall, F1
src/figures.py, src/report.py	Figures and JSON
tests/test_project.py	17 tests
docs/	Design documents and report builder

11. Input, Processing and Output

Stage	Input	Processing	Output
Register	Date 1, Date 2	ECC translation, warp	Aligned date 2, shift (dx, dy)
Normalise	Date 1, aligned date 2	Robust mean/std matching	Normalised date 2
Classify	Each date	Denoise, Lab K-Means, rules, mode filter	Label map per date
Objects	Label map	Opening, components, extent filter, Hough	Buildings, road lines
Change	Both label maps + Delta-E	Transition gate, opening, min-area	Change mask, transition table

12. Output Behaviour

A run writes classification.png, features_date1.png, features_date2.png, change.png, area.png and analysis_report.json to the output folder. Areas are computed as pixel count x GSD squared; the GSD is a command-line input (default 10 m/px, which is only a placeholder for the synthetic demo).

13. Installation and Execution

- git clone https://github.com/heyanantsingh/TerraLens && cd TerraLens
- python -m venv venv && venv\Scripts\activate (Windows)
- pip install -r requirements.txt
- python main.py --demo
- python main.py --before a.png --after b.png --gsd 10
- pytest

14. User Interaction

Command-line only: --demo, --before, --after, --gsd, --out, --seed. No GUI.

15. Testing Approach

The suite has 17 automated tests; all 17 passed on the final code. They cover: generator shape and determinism, colour rules, the mode filter, denoising, classification accuracy, superiority over the pixelwise baseline, registration accuracy, radiometric normalisation, building count, junction rejection, change quality versus the naive baseline, transition-matrix bookkeeping, mask cleaning, report keys, the CLI end to end, and the unreadable-input exit path.

16. Results and Observations

Protocol. Rules and parameters were developed on seeds 1-3. The numbers below are for seeds 11-15, which were not used during development. Every scene has 512x512 pixels, sensor noise (sigma 15 grey levels), a (4, -3) pixel mis-registration, and a gain/bias illumination change on date 2. **The scene layout and the four change events are the same for every seed**; seeds change texture, noise and building placement. The benchmark therefore measures robustness to noise and placement, not generalisation to arbitrary geometry.

Table 1. Land-cover classification (mean +/- std over 5 held-out seeds)

Method	Pixel accuracy	Mean IoU
TerraLens (denoise + Lab K-Means + rules + mode filter)	0.999 +/- 0.000	0.994 +/- 0.000
Baseline (same colour rules, pixelwise on raw image)	0.949 +/- 0.000	0.795 +/- 0.001

Table 2. Per-class IoU (mean over seeds)

Class	Bare soil	Vegetation	Water	Built-up
TerraLens	0.998	0.999	1.000	0.978
Baseline	0.935	0.936	0.835	0.473

Table 3. Change detection vs ground-truth change mask

Method	Precision	Recall	F1	IoU
TerraLens (align + normalise + class gate)	1.000 +/- 0.000	0.999 +/- 0.000	0.999 +/- 0.000	0.999 +/- 0.000
Naive (grey difference + Otsu, no alignment)	0.472 +/- 0.024	0.740 +/- 0.030	0.576 +/- 0.024	0.405 +/- 0.024

Registration. Mean error of the estimated shift: 0.09 px (std 0.03). **Buildings.** Predicted counts equalled the true counts on all 10 scene dates (5 seeds x 2 dates): 13/13 then 24/24, 14/14 then 23/23, 13/13 then 25/25, 13/13 then 23/23, 15/15 then 27/27 (predicted/true).

The change F1 is essentially identical across seeds because the geometry is fixed; the near-perfect value should be read as 'the pipeline handles this synthetic setting', not as an expected real-world accuracy.

Demo run (seed 7)

Estimated shift [3.92, -3.04] px (true 4, -3). Changed area 249.13 ha at an assumed 10 m/px. Buildings 11 to 23. Largest transitions: Vegetation -> Bare soil 118.67 ha; Bare soil -> Vegetation 55.88 ha; Water -> Bare soil 43.88 ha; Vegetation -> Built-up 30.7 ha.

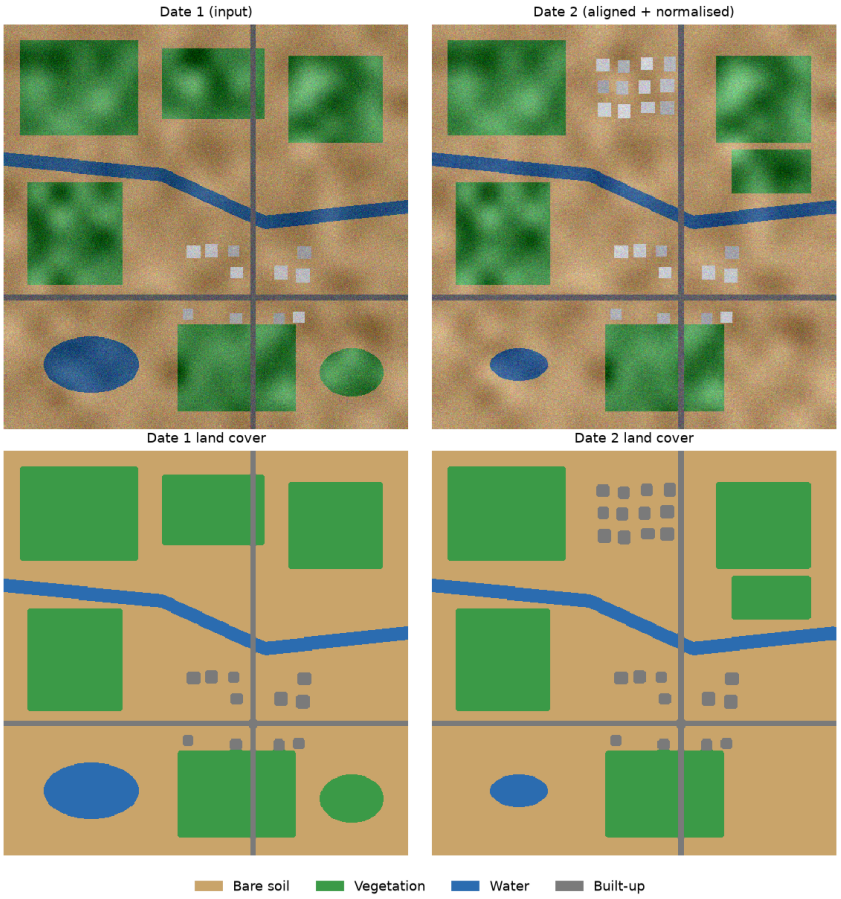


Figure 2. Inputs and land-cover maps for both dates

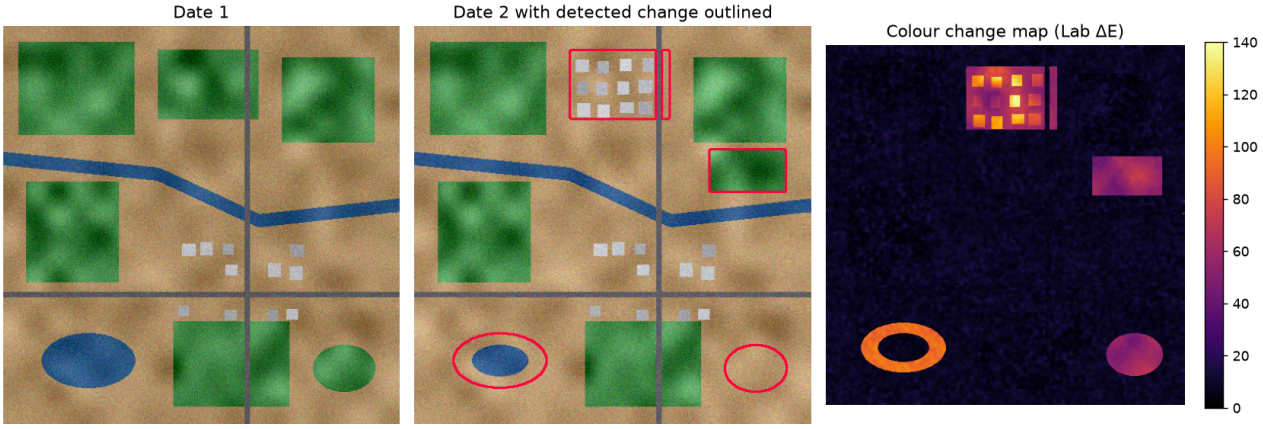


Figure 3. Detected change (red outlines) and Lab colour-difference map

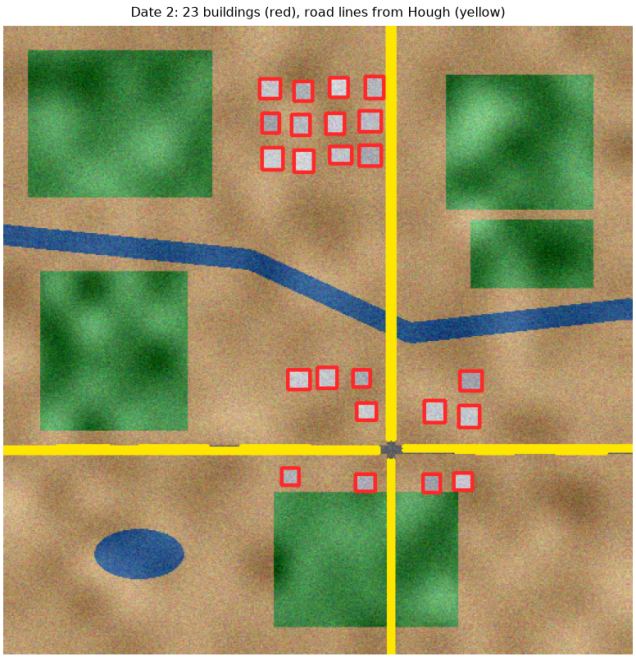


Figure 4. Buildings (red boxes) and Hough road lines (yellow) on date 2

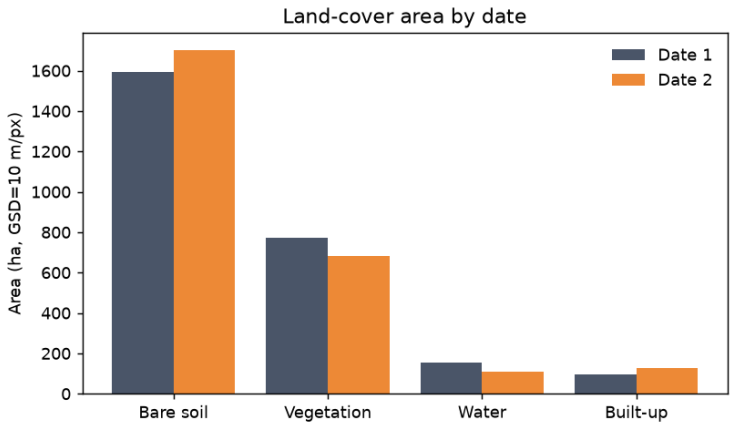


Figure 5. Land-cover area per date

17. Limitations

- **No real imagery.** All quantitative results are on synthetic scenes; real scenes add shadows, seasonal colour change, clouds, mixed pixels and sensor differences that this benchmark does not contain.
- **Fixed scene layout.** Only noise, texture and building placement vary between benchmark seeds.
- **Rules were developed on the same generator family** as the test seeds (different seeds, same generator); colour thresholds are not guaranteed to transfer to real sensors.
- **Visible bands only.** No near-infrared, so no NDVI; vegetation vs soil relies on greenness, which seasonal change can confound.
- **Translation-only registration.** Rotation, scale or terrain parallax are not handled.
- **Road output is qualitative.** Hough returns overlapping detections of the same road, so the raw segment count is not a road count; road length is not estimated.
- **Area in hectares depends on the GSD supplied** by the user; the demo's 10 m/px is a placeholder.
- Closely spaced buildings (gap under a few pixels) would merge into one blob and be undercounted.

18. Challenges Faced and Fixes

Easy synthetic data hid the difference between methods. At sensor noise sigma 7 the pixelwise baseline already scored about 99.8% accuracy on the first scene tested, so the benchmark could not separate the methods. Noise was raised to sigma 15 (a harsher, not a tuned-to-win, setting) before measuring.

Cluster merging on one seed. With plain Lab features, seed 3 merged a light-grey building cluster into soil (built-up IoU 0.69, 1 building detected instead of 13). Weighting hue over brightness fixed it; all development seeds then gave built-up IoU above 0.97.

Road junction counted as a building. The ground-truth label map itself produced one extra blob; an extent filter removed it and a test now guards this.

19. Learnings and Key Takeaways

- The naive baseline (no alignment, no normalisation, no class gating) reached change F1 of about 0.58 versus about 1.0 for the full pipeline; this comparison does not isolate which stage contributed most (no per-stage ablation was run).
- Unsupervised clustering needs a principled naming step; the choice of feature weights decided whether a whole class survived.
- A benchmark must be hard enough to separate methods; an easy one flatters everything.
- Report what was measured, on what data, and what was not tested.

20. Future Enhancements

- Evaluate on a public labelled change-detection dataset (e.g. real Sentinel-2 or aerial pairs) and report the gap to the synthetic numbers.
- Add a near-infrared band and NDVI; add shadow and cloud masks.
- Affine/homography registration (ORB + RANSAC) for larger misalignment.
- Estimate road length by skeletonisation; vary scene geometry in the generator.
- Compare against a small learned model (e.g. U-Net) as an upper-bound reference.

21. Conclusion

TerraLens shows that a carefully ordered classical pipeline - register, normalise, denoise, cluster, name, gate - can detect land-cover change without labels, and that on a harsh synthetic benchmark each stage measurably matters. Its value as a course project is the end-to-end structure, the baselines and the honest evaluation; claims about real satellite performance are explicitly left to future work.

22. References

- Evangelidis, G. and Psarakis, E. 'Parametric Image Alignment Using Enhanced Correlation Coefficient Maximization.' IEEE TPAMI, 2008.
- Tomasi, C. and Manduchi, R. 'Bilateral Filtering for Gray and Color Images.' ICCV, 1998.
- Lloyd, S. 'Least Squares Quantization in PCM.' IEEE Trans. Information Theory, 1982.
- Matas, J., Galambos, C. and Kittler, J. 'Robust Detection of Lines Using the Progressive Probabilistic Hough Transform.' CVIU, 2000.
- Otsu, N. 'A Threshold Selection Method from Gray-Level Histograms.' IEEE Trans. SMC, 1979.
- OpenCV documentation: https://docs.opencv.org/ ; scikit-learn documentation: https://scikit-learn.org/

23. Verification Note

Every number in Section 16 was produced by running the code in this repository (outputs/benchmark_results.json and outputs/analysis_report.json) and is inserted into this PDF by docs/build_report.py. The test suite result (17 passed) is from the final code. Nothing in this report was measured on real satellite data.